

Comprehensive Machine Learning Notes

Machine Learning Handbook

May 10, 2026

Contents

1	Introduction to Machine Learning	3
2	Types of Machine Learning	3
2.1	Supervised Learning	3
2.2	Unsupervised Learning	3
2.3	Reinforcement Learning	3
3	Important Python Libraries	3
4	Handling Missing Values	3
4.1	Check Missing Values	3
4.2	Fill Missing Values Using Mean	4
4.3	Drop Missing Values	4
5	Outliers Detection and Removal	4
5.1	Detect Outliers Using Boxplot	4
5.2	IQR Method	4
5.3	Python Code for IQR Method	4
5.4	Z-Score Method	5
6	Feature Scaling	5
6.1	Standardization	5
6.2	Normalization	5
7	Encoding Techniques	6
7.1	Label Encoding	6
7.2	One Hot Encoding	6
8	Train Test Split	6
9	Cross Validation	13
10	Hyperparameter Tuning	13
11	Model Evaluation Metrics	13
11.1	Regression Metrics	13
11.2	Classification Metrics	14

12 Overfitting and Underfitting	14
12.1 Overfitting	14
12.2 Underfitting	14
13 Pipeline in Machine Learning	14
14 Important Machine Learning Interview Questions	14
15 Conclusion	15

1 Introduction to Machine Learning

Machine Learning (ML) is a subset of Artificial Intelligence (AI) that allows systems to learn patterns from data and make predictions.

2 Types of Machine Learning

2.1 Supervised Learning

Uses labeled data.

Examples:

- House Price Prediction
- Spam Detection
- Salary Prediction

2.2 Unsupervised Learning

Uses unlabeled data.

Examples:

- Customer Segmentation
- Clustering

2.3 Reinforcement Learning

Learns using rewards and penalties.

3 Important Python Libraries

Library	Purpose
NumPy	Numerical Computation
Pandas	Data Handling
Matplotlib	Data Visualization
Seaborn	Statistical Visualization
Scikit-learn	Machine Learning Algorithms
TensorFlow	Deep Learning

4 Handling Missing Values

4.1 Check Missing Values

```

1 import pandas as pd
2
3 data = pd.read_csv('data.csv')
4
5 print(data.isnull().sum())

```

4.2 Fill Missing Values Using Mean

```

1 data['Age'].fillna(data['Age'].mean(), inplace=True)

```

4.3 Drop Missing Values

```

1 data.dropna(inplace=True)

```

5 Outliers Detection and Removal

Outliers are extreme values that are very different from other values.

5.1 Detect Outliers Using Boxplot

```

1 import seaborn as sns
2 import matplotlib.pyplot as plt
3
4 sns.boxplot(x=data['Salary'])
5 plt.show()

```

5.2 IQR Method

$$IQR = Q3 - Q1$$

5.3 Python Code for IQR Method

```

1 Q1 = data['Salary'].quantile(0.25)
2 Q3 = data['Salary'].quantile(0.75)
3
4 IQR = Q3 - Q1
5
6 lower_limit = Q1 - 1.5 * IQR
7 upper_limit = Q3 + 1.5 * IQR
8
9 outliers = data[
10     (data['Salary'] < lower_limit) |
11     (data['Salary'] > upper_limit)
12 ]
13
14 print(outliers)

```

```

15
16 clean_data = data[
17     (data['Salary'] >= lower_limit) &
18     (data['Salary'] <= upper_limit)
19 ]
20
21 print(clean_data)

```

5.4 Z-Score Method

$$Z = \frac{X - \mu}{\sigma}$$

```

1 from scipy import stats
2 import numpy as np
3
4 z = np.abs(stats.zscore(data['Salary']))
5
6 clean_data = data[z < 3]
7
8 print(clean_data)

```

6 Feature Scaling

Feature Scaling standardizes data ranges.

6.1 Standardization

$$Z = \frac{X - \mu}{\sigma}$$

```

1 from sklearn.preprocessing import StandardScaler
2
3 scaler = StandardScaler()
4
5 X_scaled = scaler.fit_transform(X)
6
7 print(X_scaled)

```

6.2 Normalization

$$X' = \frac{X - X_{min}}{X_{max} - X_{min}}$$

```

1 from sklearn.preprocessing import MinMaxScaler
2
3 scaler = MinMaxScaler()
4
5 X_normalized = scaler.fit_transform(X)
6
7 print(X_normalized)

```

7 Encoding Techniques

7.1 Label Encoding

```
1 from sklearn.preprocessing import LabelEncoder
2
3 encoder = LabelEncoder()
4
5 data['Gender'] = encoder.fit_transform(data['Gender'])
6
7 print(data.head())
```

7.2 One Hot Encoding

```
1 encoded_data = pd.get_dummies(data, columns=['City'])
2
3 print(encoded_data.head())
```

8 Train Test Split

```
1 from sklearn.model_selection import train_test_split
2
3 X_train, X_test, y_train, y_test = train_test_split(
4     X,
5     y,
6     test_size=0.2,
7     random_state=42
8 )
```

sectionSimple Linear Regression (SLR)

Simple Linear Regression is one of the most fundamental supervised machine learning algorithms used for predicting continuous numerical values using a single independent variable.

The main objective of Simple Linear Regression is to establish a linear relationship between the dependent variable and the independent variable.

subsectionReal World Applications

beginitemize

item Salary Prediction

item Sales Forecasting

item Temperature Prediction

item Student Marks Prediction

enditemize

subsectionAdvantages

beginitemize

item Easy to understand

item Fast training

item Interpretable model

enditemize

```

    subsectionDisadvantages
    beginitemize
item Works only for linear relationships
item Sensitive to outliers
enditemize
    subsectionAssumptions
    beginitemize
item Linear relationship between variables
item No extreme outliers
item Independent observations
enditemize
    subsectionWorking Process
    beginenumerate
item Collect dataset
item Split dataset
item Train model
item Predict output
item Evaluate model
endenumerate

```

$$Y = mX + c$$

```

1 import pandas as pd
2 import matplotlib.pyplot as plt
3
4 from sklearn.model_selection import train_test_split
5 from sklearn.linear_model import LinearRegression
6 from sklearn.metrics import r2_score
7
8 data = pd.read_csv('salary_data.csv')
9
10 X = data[['YearsExperience']]
11 y = data['Salary']
12
13 X_train, X_test, y_train, y_test = train_test_split(
14     X, y, test_size=0.2, random_state=42
15 )
16
17 model = LinearRegression()
18
19 model.fit(X_train, y_train)
20
21 predictions = model.predict(X_test)
22
23 print('R2 Score:', r2_score(y_test, predictions))
24
25 plt.scatter(X, y)
26 plt.plot(X, model.predict(X), color='red')
27 plt.show()

```

sectionMultiple Linear Regression (MLR)

Multiple Linear Regression extends Simple Linear Regression by using multiple independent variables to predict the dependent variable.

It is widely used in data science and analytics where several features influence the target variable.

subsectionReal World Applications

beginitemize

item House Price Prediction

item Stock Market Analysis

item Sales Prediction

item Medical Risk Prediction

enditemize

subsectionAdvantages

beginitemize

item Higher prediction accuracy

item Uses multiple features

item Handles complex datasets

enditemize

subsectionDisadvantages

beginitemize

item Can suffer from multicollinearity

item More computationally expensive

enditemize

subsectionAssumptions

beginitemize

item Linear relationship

item No multicollinearity

item Homoscedasticity

enditemize

$$Y = b_0 + b_1X_1 + b_2X_2 + b_3X_3$$

```
1 data = pd.read_csv('house_price.csv')
2
3 X = data[['Area', 'Bedrooms', 'Age']]
4 y = data['Price']
5
6 X_train, X_test, y_train, y_test = train_test_split(
7     X, y, test_size=0.2, random_state=42
8 )
9
10 model = LinearRegression()
11 model.fit(X_train, y_train)
12
13 predictions = model.predict(X_test)
14
15 print(predictions)
```

sectionPolynomial Linear Regression (PLR)

Polynomial Regression is used when the relationship between variables is nonlinear.

Instead of fitting a straight line, Polynomial Regression fits a curve to the dataset.

```

    subsectionApplications
    beginitemize
item Growth Prediction
item Population Forecasting
item Financial Trend Analysis
enditemize
    subsectionAdvantages
    beginitemize
item Captures nonlinear patterns
item Better flexibility
enditemize
    subsectionDisadvantages
    beginitemize
item High risk of overfitting
item Sensitive to noisy data
enditemize

```

$$Y = b_0 + b_1X + b_2X^2 + b_3X^3$$

```

1 from sklearn.preprocessing import PolynomialFeatures
2
3 data = pd.read_csv('position_salary.csv')
4
5 X = data[['Level']]
6 y = data['Salary']
7
8 poly = PolynomialFeatures(degree=3)
9
10 X_poly = poly.fit_transform(X)
11
12 model = LinearRegression()
13
14 model.fit(X_poly, y)
15
16 plt.scatter(X, y)
17 plt.plot(X, model.predict(X_poly), color='red')
18 plt.show()

```

sectionBinary Classification using Logistic Regression

Logistic Regression is one of the most popular classification algorithms used when the target variable contains only two classes.

The algorithm uses the sigmoid function to convert predictions into probabilities.

```

    subsectionApplications
    beginitemize
item Spam Detection
item Disease Prediction
item Fraud Detection
item Customer Churn Prediction
enditemize
    subsectionAdvantages

```

- beginitemize
 - item Fast and efficient
 - item Easy to interpret
 - item Works well for linearly separable data
- enditemize
 - subsectionDisadvantages
 - beginitemize
 - item Cannot handle highly complex relationships
 - item Sensitive to outliers
 - enditemize
 - subsectionSigmoid Function

$$P(Y = 1) = \frac{1}{1 + e^{-z}}$$

```

1 from sklearn.linear_model import LogisticRegression
2 from sklearn.metrics import accuracy_score
3
4 data = pd.read_csv('diabetes.csv')
5
6 X = data.drop('Outcome', axis=1)
7 y = data['Outcome']
8
9 X_train, X_test, y_train, y_test = train_test_split(
10     X, y, test_size=0.2, random_state=42
11 )
12
13 model = LogisticRegression(max_iter=1000)
14
15 model.fit(X_train, y_train)
16
17 predictions = model.predict(X_test)
18
19 print('Accuracy:', accuracy_score(y_test, predictions))

```

sectionMulticlass Classification using Decision Tree

Decision Tree is a supervised learning algorithm used for both classification and regression tasks.

It creates a tree-like structure where each internal node represents a condition and each leaf node represents a final prediction.

- subsectionApplications
 - beginitemize
 - item Loan Approval Systems
 - item Medical Diagnosis
 - item Customer Segmentation
 - enditemize
 - subsectionAdvantages
 - beginitemize
 - item Easy visualization
 - item Handles categorical data

- item No feature scaling required
- enditemize
- subsectionDisadvantages
- beginitemize
- item Overfitting problem
- item Unstable with noisy data
- enditemize

```

1 from sklearn.datasets import load_iris
2 from sklearn.tree import DecisionTreeClassifier
3 from sklearn.metrics import classification_report
4
5 iris = load_iris()
6
7 X = iris.data
8 y = iris.target
9
10 X_train, X_test, y_train, y_test = train_test_split(
11     X, y, test_size=0.2, random_state=42
12 )
13
14 model = DecisionTreeClassifier()
15
16 model.fit(X_train, y_train)
17
18 predictions = model.predict(X_test)
19
20 print(classification_report(y_test, predictions))

```

sectionK-Nearest Neighbors (KNN)

KNN is a supervised machine learning algorithm that predicts output based on the nearest neighboring data points.

- subsectionApplications
- beginitemize
- item Recommendation Systems
- item Image Recognition
- item Pattern Detection
- enditemize
- subsectionAdvantages
- beginitemize
- item Simple implementation
- item No training phase
- enditemize
- subsectionDisadvantages
- beginitemize
- item Slow for large datasets
- item Sensitive to scaling
- enditemize

```

1 from sklearn.neighbors import KNeighborsClassifier
2

```

```
3 model = KNeighborsClassifier(n_neighbors=3)
```

```
4
```

```
5 model.fit(X_train, y_train)
```

```
6
```

```
7 predictions = model.predict(X_test)
```

```
8
```

```
9 print(predictions)
```

sectionSupport Vector Machine (SVM)

Support Vector Machine is a powerful classification algorithm used for both linear and nonlinear datasets.

It finds the optimal hyperplane that maximizes the margin between classes.

subsectionApplications

beginitemize

item Face Detection

item Text Classification

item Bioinformatics

enditemize

subsectionAdvantages

beginitemize

item Effective in high-dimensional spaces

item Works well for complex datasets

enditemize

subsectionDisadvantages

beginitemize

item Computationally expensive

item Difficult for large datasets

enditemize

```
1 from sklearn.svm import SVC
```

```
2
```

```
3 model = SVC(kernel='linear')
```

```
4
```

```
5 model.fit(X_train, y_train)
```

sectionRandom Forest Classifier

Random Forest is an ensemble learning algorithm that combines multiple decision trees to improve prediction accuracy.

subsectionApplications

beginitemize

item Fraud Detection

item Recommendation Systems

item Medical Diagnosis

enditemize

subsectionAdvantages

beginitemize

item High accuracy

item Reduces overfitting

item Handles large datasets

enditemize

subsectionDisadvantages

```
beginitemize
item Slower training
item Complex interpretation
enditemize
```

```
1 from sklearn.ensemble import RandomForestClassifier
2
3 model = RandomForestClassifier(n_estimators=100)
4
5 model.fit(X_train, y_train)
```

9 Cross Validation

```
1 from sklearn.model_selection import cross_val_score
2
3 scores = cross_val_score(model, X, y, cv=5)
4
5 print(scores)
6 print(scores.mean())
```

10 Hyperparameter Tuning

```
1 from sklearn.model_selection import GridSearchCV
2 from sklearn.ensemble import RandomForestClassifier
3
4 parameters = {
5     'n_estimators': [50, 100, 200],
6     'max_depth': [3, 5, 10]
7 }
8
9 model = RandomForestClassifier()
10
11 grid = GridSearchCV(model, parameters, cv=5)
12
13 grid.fit(X, y)
14
15 print(grid.best_params_)
```

11 Model Evaluation Metrics

11.1 Regression Metrics

$$MAE = \frac{1}{n} \sum |y - \hat{y}|$$

$$MSE = \frac{1}{n} \sum (y - \hat{y})^2$$

11.2 Classification Metrics

$$Accuracy = \frac{CorrectPredictions}{TotalPredictions}$$

12 Overfitting and Underfitting

12.1 Overfitting

Model memorizes training data.

Solutions:

- Cross Validation
- Regularization
- More Data

12.2 Underfitting

Model is too simple.

Solutions:

- Increase complexity
- Add features

13 Pipeline in Machine Learning

```
1 from sklearn.pipeline import Pipeline
2 from sklearn.preprocessing import StandardScaler
3 from sklearn.linear_model import LogisticRegression
4
5 pipeline = Pipeline([
6     ('scaler', StandardScaler()),
7     ('model', LogisticRegression())
8 ])
9
10 pipeline.fit(X_train, y_train)
```

14 Important Machine Learning Interview Questions

1. What is Machine Learning?
2. Difference between AI and ML?
3. What is overfitting?
4. What is feature scaling?
5. Difference between regression and classification?

6. What is Logistic Regression?
7. What is Precision and Recall?
8. What is Confusion Matrix?

15 Conclusion

Machine Learning is one of the most powerful technologies in modern software development. Understanding preprocessing, scaling, outlier removal, regression, classification, and model evaluation is essential for building intelligent systems.